

Guía de Configuración Local

Sistema de Vacaciones — PostgreSQL + Prisma + Next.js

Junio 2026

1. Levantar PostgreSQL localmente

Hay dos opciones recomendadas. Se recomienda Docker por ser la más limpia y repetible.

Opción A — Docker (recomendada)

El proyecto ya incluye un archivo docker-compose.yml en la raíz con PostgreSQL configurado. Requisito: tener Docker Desktop instalado (docker.com/products/docker-desktop).

Pasos

1	Desde la raíz del proyecto, levantar los contenedores <code>docker compose up -d</code>
2	Verificar que el contenedor esté corriendo <code>docker ps</code>
3	Detener los contenedores cuando no los use <code>docker compose down</code>
4	Volver a levantarlos <code>docker compose up -d</code>

⚠ Nota: Los datos persisten entre reinicios gracias al volumen postgres_data definido en el docker-compose.yml. Las credenciales configuradas son: usuario postgres, contraseña postgres, base de datos vacations_db.

Opción B — Instalación nativa en Windows

El instalador oficial de PostgreSQL para Windows incluye todo lo necesario: el servidor, pgAdmin (interfaz gráfica) y las herramientas de línea de comandos como psql.

1	Descargar el instalador Vaya a postgresql.org/download/windows y descargue el instalador de PostgreSQL 17
2	Ejecutar el instalador

	Siga el asistente. Anote bien la contraseña del usuario postgres que defina.
3	Agregar psql al PATH (si el instalador no lo hizo) Vaya a Configuración → Sistema → Información → Configuración avanzada del sistema → Variables de entorno. En Variables del sistema edite Path y agregue: C:\Program Files\PostgreSQL\17\bin
4	Verificar la instalación (abra PowerShell nuevo) psql --version
5	Conectarse como superusuario psql -U postgres
6	Crear usuario admin con permisos (dentro de psql) CREATE USER admin WITH PASSWORD 'admin123'; CREATE DATABASE vacaciones OWNER admin; GRANT ALL ON SCHEMA public TO admin; GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin; ALTER USER admin CREATEDB;
7	Salir de psql \q

⚠ **Nota:** En Windows no existe sudo. Los comandos psql, npx y npm funcionan igual en PowerShell o CMD una vez que psql está en el PATH.

Opción C — Instalación nativa (macOS / Linux)

macOS con Homebrew

1	Instalar PostgreSQL brew install postgresql@17
2	Iniciar el servicio brew services start postgresql@17
3	Crear la base de datos createdb vacaciones

Ubuntu / Debian

1	Instalar PostgreSQL sudo apt install postgresql postgresql-contrib
2	Iniciar el servicio sudo systemctl start postgresql
3	Acceder como superusuario sudo -u postgres psql
4	Crear usuario y base de datos (dentro de psql) CREATE USER admin WITH PASSWORD 'admin123'; CREATE DATABASE vacaciones OWNER admin;

2. Importar el schema dump (producción)

Ya se cuenta con el archivo `schema_dump.sql` que contiene únicamente la estructura (tablas, tipos, funciones, restricciones). Siga estos pasos para aplicarlo a la base de datos local.

2.1 Verificar el archivo `.env` del proyecto

El archivo `.env` ya existe en `vacation_system/.env`. Asegúrese de que la `DATABASE_URL` apunte a la base de datos local con las credenciales correctas del `docker-compose.yml`:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/vacations_db
NEXTAUTH_SECRET=tu secreto aquí NEXTAUTH_URL=http://localhost:3000
```

⚠ Nota: Si usa instalación nativa de Windows con el usuario `admin` creado en el paso anterior, la URL sería: `postgresql://admin:admin123@localhost:5432/vacaciones`

2.2 Aplicar el dump con `psql`

El archivo `schema_dump.sql` debe estar en `docs/db/` en la raíz del repositorio. Ejecute el siguiente comando desde la carpeta `vacation_system/`:

```
psql -h localhost -U postgres -d vacations db -f ../docs/db/schema_dump.sql
```

Se le pedirá la contraseña (`postgres`). El comando crea todas las tablas, tipos y funciones en la base de datos.

2.3 Solución: error de permisos en el esquema `public`

Si al correr el dump aparece el error 'permiso denegado al esquema `public`', conéctese con el superusuario y otorgue los permisos necesarios:

```
psql -U postgres -d vacations db
```

Dentro de `psql` ejecute:

```
GRANT ALL ON SCHEMA public TO admin; GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA
public TO admin; ALTER USER admin CREATEDB;
```

Luego salga con `\q` y vuelva a correr el dump.

2.4 Verificar la importación

```
psql -h localhost -U postgres -d vacations db -c "\dt public.*"
```

Debería ver la lista de tablas: `admin_nombramiento`, `ciclo`, `curso`, `curso_grupo`, `profesor`, `proyecto`, etc.

3. Configurar Prisma

Prisma ya está incluido en el `package.json` del proyecto (versión 7.8.0). En Prisma 7 la URL de conexión ya no va en `schema.prisma` sino en `prisma.config.ts`, que ya viene configurado en el proyecto. Todos los comandos se ejecutan desde la carpeta `vacation_system/`.

3.1 Verificar el schema.prisma

Abra `vacation_system/prisma/schema.prisma` y confirme que el `datasource` NO tenga la línea `url`, ya que en Prisma 7 esa configuración vive en `prisma.config.ts`:

```
datasource db {
  provider = "postgresql"
}
```

⚠ Nota: Si el bloque `datasource` tiene `url = env("DATABASE_URL")`, elimine esa línea. Si dice `provider = "prisma-postgres"`, cámbielo a `"postgresql"`.

3.2 Introspeccionar la base de datos (solo la primera vez)

Con el dump ya aplicado, corra el siguiente comando para que Prisma lea la estructura de la base de datos y genere los modelos en `schema.prisma`:

```
npx prisma db pull
```

Este paso solo lo ejecuta quien configura el proyecto por primera vez. El `schema.prisma` resultante se commitea al repositorio para que los demás compañeros ya lo tengan.

3.3 Corrección del enum `jornada_fraccion_tipo`

Prisma no puede representar valores de enum que empiezan con número o contienen caracteres especiales como `/`. Al correr `db pull`, los valores de ese enum quedan comentados y hay que corregirlos manualmente en `prisma/schema.prisma`. Busque el enum `jornada_fraccion_tipo` y reemplácelo con:

```
enum jornada_fraccion_tipo {
  CERO @map("0")
  UN_OCTAVO @map("1/8")
  UN_CUARTO @map("1/4")
  TRES_OCTAVOS @map("3/8")
  UN_MEDIO @map("1/2")
  CINCO_OCTAVOS @map("5/8")
  TRES_CUARTOS @map("3/4")
  SIETE_OCTAVOS @map("7/8")
  UNO @map("1")
}
```

El `@map` le indica a Prisma cómo se almacena el valor en la base de datos, mientras que el nombre del lado izquierdo es el que se usa en el código TypeScript.

3.4 Generar el cliente de Prisma

Una vez corregido el schema, genere el cliente desde `vacation_system/`:

```
npx prisma generate
```

A partir de aquí puede importar `PrismaClient` en su código Next.js para hacer consultas.

3.5 Abrir Prisma Studio (opcional)

Prisma incluye una interfaz visual para explorar y editar datos directamente en el navegador:

```
npx prisma studio
```

⚠ Nota: Prisma Studio abre en `http://localhost:5555` por defecto y es muy útil durante el desarrollo.

4. Insertar filas de prueba

A continuación se muestran ejemplos para insertar registros en las tablas más importantes del sistema. Puede ejecutar estos comandos directamente en psql o usando Prisma Studio.

4.1 Ejecutar psql interactivo

Desde cualquier carpeta:

```
psql -h localhost -U postgres -d vacations_db
```

4.2 Insertar un ciclo académico

La tabla ciclo almacena los períodos (1 o 2) y el año. Es requisito de casi todo lo demás.

```
INSERT INTO ciclo (periodo, anno) VALUES (1, 2026), (2, 2026) RETURNING *;
```

4.3 Insertar profesores

La tabla profesor requiere al menos: identificacion, nombre, apellido1, estado y regimen_salarial. El régimen acepta los valores: Salario Compuesto, Salario Global o NaN.

```
INSERT INTO profesor ( identificacion, nombre, apellido1, apellido2, email_institucional, estado, regimen_salarial ) VALUES ('11111111', 'Ana', 'Pérez', 'Mora', 'aperez@institucion.ac.cr', 'Activo', 'Salario Global'), ('22222222', 'Carlos', 'Rodríguez', 'Vega', 'crodriguez@institucion.ac.cr', 'Activo', 'Salario Compuesto') RETURNING id, nombre, apellido1;
```

4.4 Insertar un departamento

```
INSERT INTO departamento (nombre, estado) VALUES ('Cómputo', 'Activo') RETURNING id;
```

4.5 Insertar un curso

Use el id devuelto en el paso anterior como id_departamento.

```
INSERT INTO curso (sigla, nombre, id_departamento, creditaje, horas, estado) VALUES ('IC1101', 'Progr. en Computadores', 1, 3, 4, 'Activo') RETURNING *;
```

4.6 Insertar un grupo de curso

Asume que el ciclo tiene id=1 y el curso tiene id=1.

```
INSERT INTO curso_grupo (id_curso, id_ciclo, numero, estado) VALUES (1, 1, 1, 'Activo') RETURNING *;
```

4.7 Insertar un nombramiento de curso

Asigna al profesor con id=1 al grupo con id=1, con 4 horas de carga.

```
INSERT INTO curso_nombramiento (id_grupo, id_profesor, carga_horas) VALUES (1, 1, 4) RETURNING *;
```

⚠ Nota: Respete el orden de inserción: ciclo → departamento → profesor → curso → curso_grupo → nombramiento. Las llaves foráneas con ON DELETE RESTRICT impiden insertar registros huérfanos.

4.8 Verificar los datos insertados

Ver todos los profesores activos:

```
SELECT id, nombre || ' ' || apellido1 AS nombre_completo, regimen_salarial FROM profesor WHERE estado = 'Activo';
```

Ver grupos con su curso y ciclo:

```
SELECT cg.id, c.sigla, c.nombre, ci.anno, ci.periodo FROM curso_grupo cg JOIN curso c ON c.id = cg.id_curso JOIN ciclo ci ON ci.id = cg.id_ciclo;
```

5. Resumen del flujo completo

⚠ Nota: Los comandos psql, npm y npx funcionan igual en Windows, macOS y Linux. En Windows no existe sudo; use PowerShell o CMD con el usuario postgres (instalación nativa) o cualquier usuario si usa Docker.

5.1 Primera configuración (quien levanta el proyecto por primera vez)

#	Paso	Comando / acción
1	Levantar PostgreSQL (desde raíz)	<code>docker compose up -d</code>
2	Verificar .env en vacation_system/	<code>DATABASE_URL=postgresql://postgres:postgres@localhost:5432/vacations_db</code>
3	Instalar dependencias (desde vacation_system/)	<code>npm install</code>
4	Aplicar dump (desde vacation_system/)	<code>psql -h localhost -U postgres -d vacations_db -f ../docs/db/schema_dump.sql</code>
5	Verificar schema.prisma (sin url en datasource)	<code>provider = "postgresql" - sin línea url</code>
6	Introspeccionar BD	<code>npx prisma db pull</code>
7	Corregir enum jornada_fraccion_ti	Ver sección 3.3

	po en schema.prisma	
8	Generar cliente	<code>npx prisma generate</code>
9	Commitear schema.prisma al repositorio	<code>git add prisma/schema.prisma && git commit</code>

5.2 Configuración para el resto del equipo (schema ya commiteado)

#	Paso	Comando / acción
1	Clonar el repositorio	<code>git clone <url-del-repo></code>
2	Levantar PostgreSQL (desde raíz)	<code>docker compose up -d</code>
3	Crear .env en vacation_system/ con la URL correcta	<code>DATABASE_URL=postgresql://postgres:postgres@localhost:5432/vacations_db</code>
4	Instalar dependencias (desde vacation_system/)	<code>npm install</code>
5	Aplicar dump (desde vacation_system/)	<code>psql -h localhost -U postgres -d vacations_db -f ../docs/db/schema_dump.sql</code>
6	Generar cliente Prisma	<code>npx prisma generate</code>